Data Analytics Capstone

Predicting Service Level Agreement Breach in IT Incident Management: An Analysis of Ticket-Level Drivers in an Enterprise ServiceNow Event Log

Final Report

Binesh Balakrishnan

Walsh College

QM640: Data Analytics Capstone

Ms Keya Choudhury Ganguli

Term 3

November 10, 2026

GitHub Repository (Data and Code)

https://github.com/binesh29/QM640_data_analytics_capstone

This public repository contains the original UCI data file (data/raw), the one-row-per-incident file used in the analysis (data/processed), six Jupyter notebooks that generate all tables and figures in this report (notebooks/01 to 06), the Python code and deployment prototype (src/ and app/), all numeric outputs in JSON format (results/), and the figure files (figures/). The README gives installation steps for the pinned library versions and the order in which to run the notebooks, and Appendix B maps each notebook to its results.

Table of Contents

Abstract....4
Introduction....6
Background and Context....6
Problem Statement....7
Purpose of the Study....7
Research Problems / Research Questions....7
Contributions and Expected Value....8
Literature Review....8
Literature Review Approach....8
Summary of Key Literature....9
Materials and Method....12
Data Source and Acquisition....12
Dataset Overview and Inclusion and Exclusion Criteria....13
Data Dictionary....14
Exploratory Data Analysis....15
Research Hypotheses....19
Minimum Sample Size....20
Statistical Methods....21
Database Set-Up and Data Access....22
Data Splitting....22
Model Selection and Justification....23
Training and Test Accuracy and Comparison With Current Methods....23
Architecture Diagram/Workflow....24
System Overview....24
Architecture Diagram....25
Workflow Components....26
Tools and Technologies....28
Results....29
Model Performance....29
Visual Evidence....30
Results by Research Question....34
Overall Interpretation....36
Practical Significance....37
Implementation and User Benefit....37
Deployment Approach....37
System Integration....38
User Interaction....38
Benefits to Users....39
Example Use Case....39
Limitations and Further Improvements....40
Limitations....40
Impact of Limitations....40
Future Improvements....41
Future Scope....41
References....42
Appendix A: Sample Size Code....47
Appendix B: Notebook Map and Run Order....48
Appendix C: Running the Deployment Prototype....48

Abstract

Service level agreement (SLA) breaches in information technology (IT) incident management are counted after the fact, so service desks escalate tickets by priority and intuition rather than evidence. This study asks which ticket-level factors are associated with SLA breach and whether breach risk can be predicted when a ticket is logged. The approach combines statistics with machine learning (ML): χ^2 tests, multivariable logistic regression, and Mantel–Haenszel pooling test priority, reassignment, and knowledge-base use (RQ1 to RQ3), and a priority-only baseline is compared with regularised logistic regression, random forest, and extreme gradient boosting (XGBoost) by the DeLong test (RQ4). The data are a public ServiceNow event log from the University of California, Irvine (UCI) Machine Learning Repository: 141,712 audit events collapsed to N = 24,918 incidents, the unit of analysis, opened between 29 February 2016 and 16 February 2017, of which 36.6% breached. All work used Python with pandas, statsmodels, scikit-learn, and XGBoost on a central processing unit (CPU) in Google Colab and on a laptop; no graphics processing unit (GPU) was needed. All four null hypotheses were rejected at $\alpha = .05$. Critical and High incidents breached 98.1% and 99.5% of the time against 15.9% for Low ones (Cramér's $V = 0.228$). Each reassignment multiplied the adjusted odds of breach by 1.95. The raw knowledge-base effect (odds ratio 6.24) shrank to 1.17 once a March 2016 process change was accounted for. On a time-ordered test set of 7,476 incidents, a random forest using 17 attributes known at logging reached an area under the receiver operating characteristic curve (AUC) of 0.785, against 0.552 for the priority rule ($z = 38.45$, $p < .001$); flagging the 20% highest-risk tickets caught 46.6% of breaches at 2.3 times the base rate. A working prototype stores the data in SQLite, serves recalibrated risk scores through a representational state transfer (REST) application programming interface (API), and ranks the queue in a dashboard, so a

ServiceNow business rule can score each new ticket and duty managers can act before the SLA clock runs out.

Introduction

Every enterprise information technology (IT) service desk commits to resolution times in a service level agreement (SLA) and reports each month how often it missed them, after the damage is done. This study explains and predicts SLA breach from the ticket itself, using real ServiceNow data.

Background and Context

Within IT service management (ITSM), incident management is the process that brings service back after an unplanned outage. In ServiceNow, a new incident is recorded, prioritised on the basis of impact and urgency, sent to a support group, worked on, resolved, and closed, and the platform stores each change in an audit trail. The IT Infrastructure Library (ITIL) sets out this lifecycle; organisations that follow it gain service quality but still find it hard to pin down what drives process performance (Marrone & Kolbe, 2011).

In managed services, an SLA sets target resolution times for each priority. Those affected are the end users who lose service, the analysts and resolver groups who work the tickets, the service delivery manager who answers for SLA performance, and the paying client. A breach therefore has both an operational and a commercial cost.

Two behaviours are blamed for delay: reassignment between groups, which routing research treats as waste (Shao et al., 2008), and whether recorded knowledge is used. Artificial intelligence for IT operations (AIOps) promises to anticipate such problems from operational data (Dang et al., 2019), and predictive process monitoring forecasts case outcomes from event logs (Teinemaa et al., 2019; van der Aalst, 2016). Missing is a test of the factors managers act on against the contractual outcome, and a prediction early enough to change it.

Problem Statement

Service desks escalate by priority without evidence on what drives SLA breach, and they find at-risk tickets only when little SLA time remains. The objective of this study is to explain and predict Y = SLA breach (1 = breached, 0 = met) for each incident from X = priority, reassignment count, knowledge-base use, and 17 attributes captured at logging, controlling for service category, across 24,918 incidents opened between 29 February 2016 and 16 February 2017 in one ServiceNow instance. Success means rejecting each null hypothesis at $\alpha = .05$ with an effect size (Cramér's V, odds ratio) for RQ1 to RQ3, and, for RQ4, a test-set area under the receiver operating characteristic (ROC) curve (AUC) significantly above a priority-only model (DeLong test), supported by precision and recall at a fixed review capacity and the Brier score.

Purpose of the Study

The study explains how strongly priority, reassignment, and knowledge-base use are associated with breach after adjustment (RQ1 to RQ3), and classifies each new incident by breach risk at logging with machine learning (ML), comparing regularised logistic regression, random forest, and extreme gradient boosting (XGBoost) with the priority rule by AUC (RQ4). The best model is then built into a working scoring service and dashboard.

Research Problems / Research Questions

All four questions share the same dependent variable, SLA breach.

Research Question 1 (RQ1): Does the SLA breach rate differ significantly across the four priority levels (Critical, High, Moderate, Low)?

Research Question 2 (RQ2): Is the number of reassignments associated with the likelihood of SLA breach after controlling for priority and service category?

Research Question 3 (RQ3): Is knowledge-base use associated with the likelihood of SLA breach after controlling for priority and service category?

Research Question 4 (RQ4): Can an ML classifier that uses only attributes available when a ticket is logged predict SLA breach with a significantly higher AUC than a priority-only baseline?

Contributions and Expected Value

Practical contribution. Four managerial beliefs become tested, quantified statements, and each new ticket receives a risk score with a daily review rule. **Technical contribution.** Breach is predicted at logging with a leakage-free feature set and time-ordered validation, and the study shows how a process change distorts tests if ignored. **Value to stakeholders.** Managers gain evidence for routing and escalation, resolvers earlier warning, and clients fewer breaches; the method uses fields every ServiceNow instance records.

Literature Review

Literature Review Approach

Google Scholar, Crossref, arXiv, and the ACM, IEEE, and Springer libraries were searched with terms such as “ticket routing”, “SLA violation prediction”, “predictive process monitoring”, “deep learning tabular data”, and “large language model incident”. Peer-reviewed papers, two well-cited preprints, and methods texts were included if they addressed IT operations, event-log prediction, deep-learning (DL) or large language model (LLM) methods, or the techniques used here. Table 1 maps 15 core sources to the research questions and design choices.

Summary of Key Literature

Table 1

Literature Relevance Matrix

Author (Year)	Domain / context	Dataset / setting	Method(s)	Key findings	How it supports the RQs / project decisions
Marrone & Kolbe (2011)	ITSM frameworks	Survey of ITIL-adopting organisations	Empirical survey	ITIL brings benefits, but measuring process performance stays hard	Motivates measuring SLA drivers (all RQs)
Shao et al. (2008)	Ticket routing	Problem-ticket transfer histories	Resolution-sequence mining (Markov model)	Mined routing cuts transfers needed to reach a resolver	Treats reassignment as a cost driver (RQ2)
Botezatu et al. (2015)	Dispatching tickets	Corporate incident data	Clustering on several ticket views	Better first-time dispatch to the right group	First assignment group as a predictor (RQ2, RQ4)
Leitner et al. (2010)	Service computing	Composite web services	Runtime monitoring with ML prediction	SLA violations can be predicted and prevented during execution	SLA breach as a prediction target (RQ4)
Amaral et al. (2018a, 2019)	Process mining	The ServiceNow log analysed here	Attribute selection (filter, wrapper)	Choosing attributes sharpens completion-time forecasts	Same log, but breach classification untested (RQ4)
Teinemaa et al. (2019)	Predictive process monitoring	Nine real event logs (24 tasks)	Benchmark of 11 outcome-prediction methods	No method dominates; tree ensembles strong	Outcome-prediction design; model shortlist (RQ4)
Tax et al. (2017)	Predictive monitoring (DL)	Public event logs	Long short-term memory (LSTM) networks	LSTMs predict next events and remaining time of running cases	DL needs event sequences, absent at logging (RQ4)
Evermann et al. (2017)	Process prediction (DL)	Public challenge event logs	Recurrent neural networks	DL learns process behaviour without hand-made features	Considered and rejected for at-logging data (RQ4)
Bukhsh et al. (2021)	Predictive monitoring (DL)	Public event logs	Transformer network	Attention models match or beat LSTMs on sequence tasks	Future scope: in-flight risk updates
Grinsztajn et al. (2022); Shwartz-Ziv & Armon (2022)	ML on tabular data	Benchmark tabular datasets	Tree ensembles versus DL	Tree ensembles match or beat DL on typical tabular data	Choice of random forest and XGBoost over DL (RQ4)
Ahmed et al. (2023)	Cloud incidents (LLM)	40,000+ Microsoft incidents	Fine-tuned generative LLMs	LLMs can draft root-cause and mitigation text for engineers	LLM assistant layer in future scope
Chen et al. (2024)	Cloud incidents (agentic LLM)	Microsoft production incidents	RCACopilot: tool-collecting agent plus LLM	Automated data collection with LLM reasoning speeds root-cause analysis	Agent that acts on high-risk tickets (future scope)

Author (Year)	Domain / context	Dataset / setting	Method(s)	Key findings	How it supports the RQs / project decisions
Yao et al. (2023)	LLM agents	Question answering and decision tasks	ReAct: interleaved reasoning and tool actions	Reasoning-plus-acting agents outperform reasoning-only prompts	Pattern for a triage agent calling the risk-scoring service
Kaufman et al. (2012)	Applied data mining	Prediction contests and industry cases	Formalises leakage	Information unavailable at prediction time overstates accuracy	Excludes counts that accrue after logging (RQ4)
DeLong et al. (1988); Saito & Rehmsmeier (2015)	Classifier evaluation	Clinical and imbalanced data	Correlated-AUC test; precision–recall analysis	Paired AUC test; precision–recall curves more informative under imbalance	H_{04} test; precision–recall reporting (RQ4)

Note. Key findings are paraphrased from each source. DL = deep learning; LLM = large language model; ITIL = IT Infrastructure Library.

Routing and reassignment in ITSM. Ticket-handling studies treat each transfer as wasted time. Shao et al. (2008) learn from past transfer sequences which group should receive a ticket next, and Botezatu et al. (2015) cluster tickets so that the first assignment is correct. Both measure transfers or dispatch accuracy rather than the SLA result, and both need text or sequence data; ITIL users also report that performance drivers are hard to measure (Marrone & Kolbe, 2011). None quantifies how much one reassignment raises the odds of breach at equal priority, which RQ2 does.

Predicting outcomes from event logs. Predictive process monitoring estimates how an unfinished case will end (van der Aalst, 2016). Benchmarking 11 techniques on 24 tasks, Teinemaa et al. (2019) found no universal winner, with tree ensembles near the top, and Leitner et al. (2010) forecast web-service SLA violations from runtime data. The present log has been studied only for completion time (Amaral et al., 2018a, 2019). In-flight methods use events such as reassignments that have already occurred, so this study predicts at logging, before the routing choice.

Deep learning for process data. Recurrent networks predict the next activity and remaining time of running cases (Evermann et al., 2017; Tax et al., 2017), and a transformer can

match or beat them (Bukhsh et al., 2021). These models need event sequences, but at logging an incident has one record. For single-row tabular data with many categorical fields, two benchmarks find that tree ensembles match or outperform DL with far less tuning (Grinsztajn et al., 2022; Shwartz-Ziv & Armon, 2022). RQ4 therefore uses random forest and XGBoost, and a central processing unit (CPU) suffices without a graphics processing unit (GPU).

Large language models and agents in operations. Ahmed et al. (2023) fine-tune generative models on more than 40,000 cloud incidents to draft root-cause and mitigation suggestions, and Chen et al. (2024) pair automated evidence collection with LLM reasoning in RCACopilot, an agentic system built on the reasoning-plus-tool-use pattern of ReAct (Yao et al., 2023). These systems diagnose incidents from free text rather than estimating breach probability, so they complement a risk score, which can decide which tickets an agent or engineer should examine first; this is the LLM role in this study's future scope. The log has no free text, so LLM methods could not be applied directly.

Validity and evaluation. Leakage, using information unavailable at prediction time, inflates accuracy (Kaufman et al., 2012), so counts that grow after logging are excluded. For an imbalanced outcome, AUC summarises ranking (Fawcett, 2006), precision–recall metrics describe the minority class (Saito & Rehmsmeier, 2015), the Brier score checks probabilities (Brier, 1950), and two AUCs on one test set are compared by the DeLong et al. (1988) test. Time-ordered data call for order-preserving validation such as forward chaining (Bergmeir & Benítez, 2012), and distorted probabilities can be repaired by Platt scaling (Niculescu-Mizil & Caruana, 2005). Logistic regression (Hosmer et al., 2013) and Mantel–Haenszel pooling (Mantel & Haenszel, 1959) adjust for confounding, and the Holm (1979) method controls pairwise error.

Gap addressed. Existing work improves routing, predicts case outcomes while cases run, and uses LLMs for diagnosis. None of the studies reviewed tests priority, reassignment, and knowledge use against the SLA outcome in an ITSM log, and none compares a prediction made at logging with the priority rule that service desks rely on. This study addresses both points and adds a scoring service that can be deployed.

Materials and Method

Data Source and Acquisition

The study uses the Incident Management Process Enriched Event Log, which Amaral et al. (2018b) placed in the University of California, Irvine (UCI) Machine Learning Repository (https://archive.ics.uci.edu/dataset/498/incident+management+process+enriched+event+log; https://doi.org/10.24432/C57S4H). The donors extracted this audit trail from a real IT company's ServiceNow platform, added fields from its database, and anonymised it. The data arrived as one comma-separated values (CSV) file, so no web scraping was involved. Because the Creative Commons Attribution 4.0 (CC BY 4.0) licence allows redistribution with credit, the original file is kept unchanged in the project repository (Figure 1 and Figure 2).

Figure 1

Sample of the Raw Event Log: Six Events and Eight of the 36 Columns

number	incident_state	reassignment_count	made_sla	opened_at	priority	knowledge	resolved_at
INC0000045	New	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Resolved	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Resolved	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Closed	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000047	New	0	true	29/2/2016 04:40	3 - Moderate	true	1/3/2016 09:52
INC0000047	Active	1	true	29/2/2016 04:40	3 - Moderate	true	1/3/2016 09:52

Note. The same incident number repeats across rows as its state changes. From Incident Management Process Enriched Event Log [Data set], by C. Amaral, M. Fantinato, and S. Peres, 2018, UCI Machine Learning Repository (https://doi.org/10.24432/C57S4H). CC BY 4.0.

Figure 2

Folder Layout of the Project Repository

QM640_data_analytics_capstone/	
├── README.md · requirements.txt	How to run; pinned package versions
├── data/raw/incident_event_log.csv	UCI source file, unchanged (141,712 rows)
├── data/processed/incidents.csv	One row per incident (24,918 rows)
├── notebooks/	
│ ├── 01_data_preparation.ipynb	Load, validate, collapse, derive
│ ├── 02_cleaning_and_eda.ipynb	Cleaning checks and EDA figures
│ ├── 03_rq1_rq3_hypothesis_tests.ipynb	χ^2 tests, logistic regression, Mantel–Haenszel
│ ├── 04_rq4_prediction_models.ipynb	Baseline, LR, RF, XGBoost, DeLong test
│ ├── 05_extended_evaluation.ipynb	Train vs test, forward-chaining CV, calibration
│ └── 06_deployment_prototype.ipynb	Database, saved model, batch scoring, API
├── src/	sla_model, build_database, train_model, score_batch
├── app/	api.py (FastAPI), dashboard.py (Streamlit)
├── figures/ · results/	Report figures (PNG); numeric outputs (JSON)
└── docs/	Synopsis, interim and final reports

Note. Layout of https://github.com/binesh29/QM640_data_analytics_capstone. Notebook 06 regenerates the SQLite database and the saved model, so neither is kept in the repository.

Dataset Overview and Inclusion and Exclusion Criteria

The file has 141,712 rows and 36 columns; a row is written at every update, so each incident occupies 5.7 rows on average. To avoid counting incidents repeatedly, the log was reduced to one row per incident, the unit of analysis: 24,918 incidents opened from 29 February 2016 to 16 February 2017 (98.1% in March to May 2016). Each incident keeps its first record, known at creation (RQ4), and its last record, with the outcome and accumulated counts (RQ1 to RQ3). sla_breach = 1 when made_sla is false in the last record, for 9,115 incidents (36.6%); median resolution is 1.1 hours when made_sla is true and 201.7 hours when false, confirming that true denotes a met SLA.

Inclusion criteria. All incidents were kept, since each has a known outcome and the data are 6.3 times the required sample; five events with an undocumented state code (−100) were kept because they do not change the outcome. **Exclusion criteria.** Five columns that are 98% to 100% empty were dropped, apart from a missing-configuration-item flag; analyst identifiers were dropped for privacy; and information that exists only after logging (reassignment, reopen, and update counts, resolution and closure times, closure code) was kept out of the RQ4 model to avoid leakage. No incident was dropped as an outlier.

Data Dictionary

Table 2 defines the variables used, with their type, range, handling of missing values, and role. All 36 source columns are listed in the repository README.

Table 2

Data Dictionary (Incident Level)

Variable	Definition	Type	Values / range	Missing values	Role
number	Incident identifier	String	24,918 unique	None	Key
sla_breach (derived)	Equals 1 when the last record has made_sla = false	Binary	0 / 1; mean 0.366	None	Y, all RQs
priority	Derived by the platform from impact × urgency	Ordinal	1 Critical – 4 Low	None	X (RQ1); control
reassignment_count	Number of changes of group or analyst	Integer	0–27; median 0	None	X (RQ2); excluded from RQ4
knowledge	Knowledge-base article used	Boolean	true / false	None	X (RQ3); RQ4 sensitivity
category	First-level affected service	Nominal	52 levels (final); 57 at logging; 31 groups	0.2% at logging; “Missing” level	Control (RQ2–RQ3); RQ4
subcategory	Second-level service	Nominal	246 labels	0.3%; “Missing” level	RQ4
u_symptom	User-reported symptom	Nominal	500 labels	23.0%; “Missing” level	RQ4
assignment_group	First support group	Nominal	69 labels	3.6%; “Missing” level	RQ4
impact, urgency	Impact and urgency of the incident	Ordinal	1 High – 3 Low	None	RQ4

Variable	Definition	Type	Values / range	Missing values	Role
location, contact_type, notify	Affected location; channel; notification	Nominal	219; 5; 2 levels	0.2% (location); none	RQ4
u_priority_confirmation	Priority double-checked	Boolean	true / false	None	RQ4
cmdb_ci_missing (derived)	No configuration item recorded	Binary	0 / 1	None	RQ4
caller_freq (derived)	Training-period incidents of the caller, log(1 + n)	Numeric	≥ 0	Unseen caller = 0	RQ4
hour_sin, hour_cos, weekend, day of week (derived)	Cyclical hour, weekend flag, and weekday of opened_at	Numeric / binary	−1 to 1; 0 / 1; 0–6	None	RQ4
opened_at	Opening time	Datetime	29 Feb 2016 – 16 Feb 2017	None	Split order; features
resolution_hours (derived)	resolved_at − opened_at	Continuous	0 – 8,070 h	1,556 left missing	Descriptive only

Note. Label counts and missing shares are at logging (first record) unless marked final; variables with the log_ prefix in the code are first-record values. Y = dependent variable; X = independent variable. Adapted from Amaral et al. (2018b).

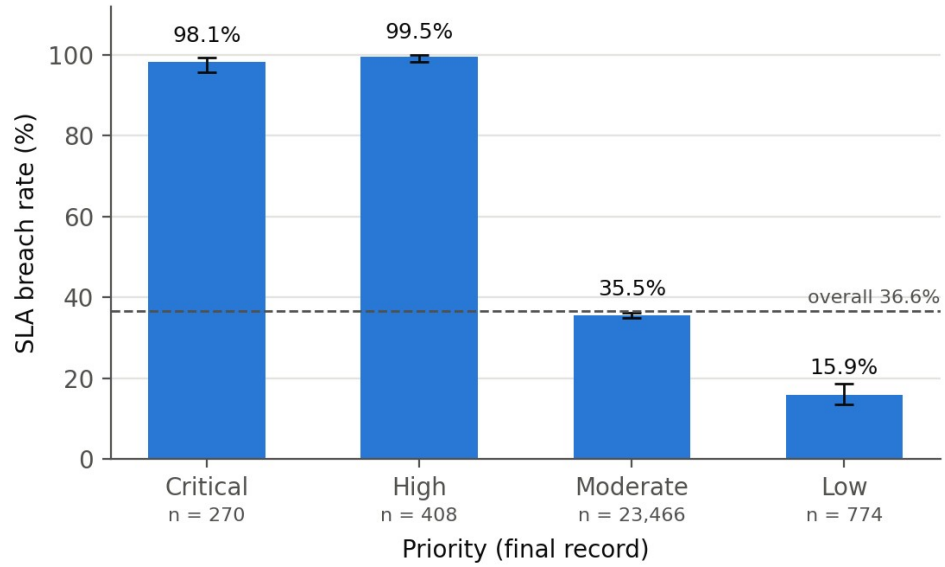
Exploratory Data Analysis

Exploratory data analysis (EDA) showed long right tails: a typical incident is never reassigned and is resolved within 22.1 hours, but mean resolution is 178.2 hours. Medians, rates, and odds ratios are therefore used instead of means, and no method assumes normality. Four findings guided the analysis.

Priority. Nearly every Critical and High incident breaches (98.1% and 99.5%), compared with 35.5% of Moderate and 15.9% of Low incidents (Figure 3). Because Moderate tickets make up 94.2% of all incidents, they produce 91.3% of the breaches, so the greatest business impact, and the greatest scope for a better rule, sits within one priority level.

Figure 3

Breach Rate for Each Priority Level

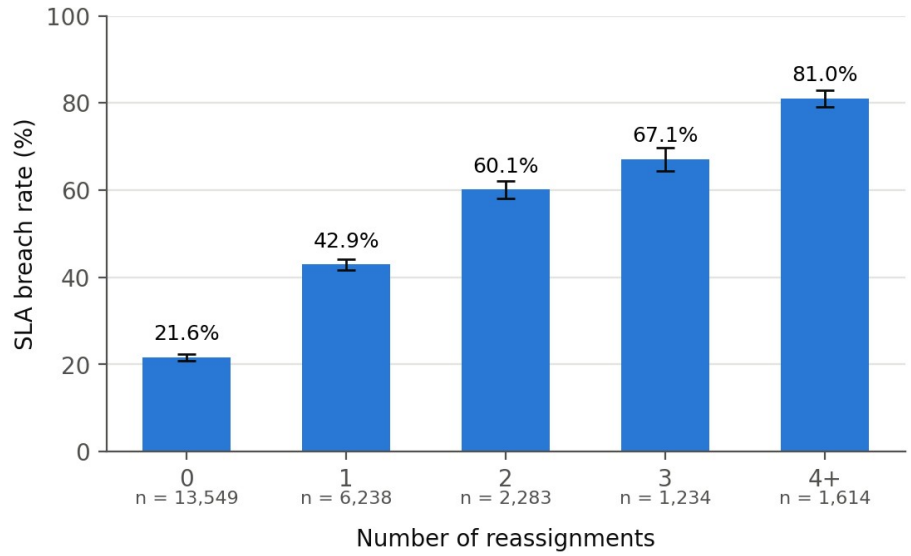


Note. Error bars show Wilson 95% confidence intervals, and the dashed line marks the overall rate of 36.6%. *n* = 270, 408, 23,466, and 774.

Reassignment. The breach rate rises with every hand-off, from 21.6% for incidents never reassigned to 42.9% after one, 60.1% after two, and 81.0% after four or more (Figure 4). Reassigned incidents are 45.6% of the total but 67.9% of breaches.

Figure 4

Breach Rate by Reassignment Count

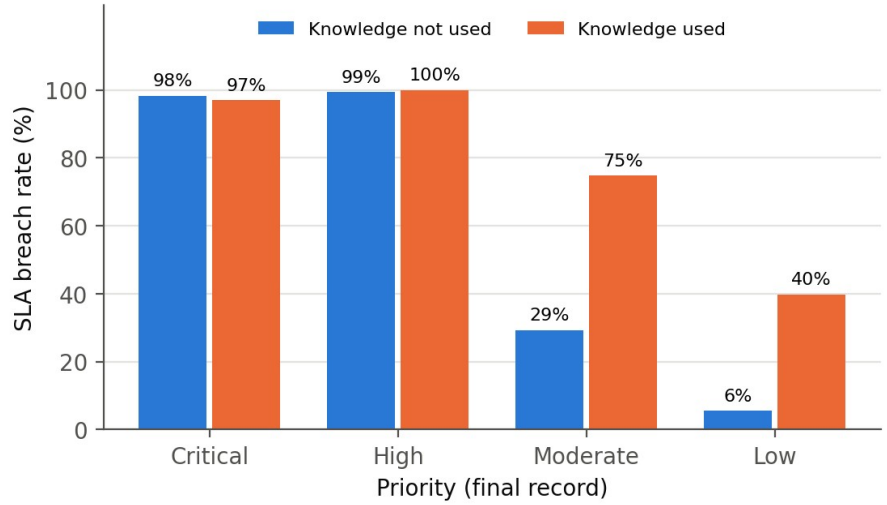


Note. Wilson 95% confidence intervals are shown as error bars.

Knowledge-base use. In the Moderate and Low groups, incidents where a knowledge article was used breached much more often (75% against 29%, and 40% against 6%; Figure 5). It is hard to believe that consulting an article causes a breach, so the timing of knowledge use was investigated next.

Figure 5

Breach Rate With and Without Knowledge-Base Use, by Priority

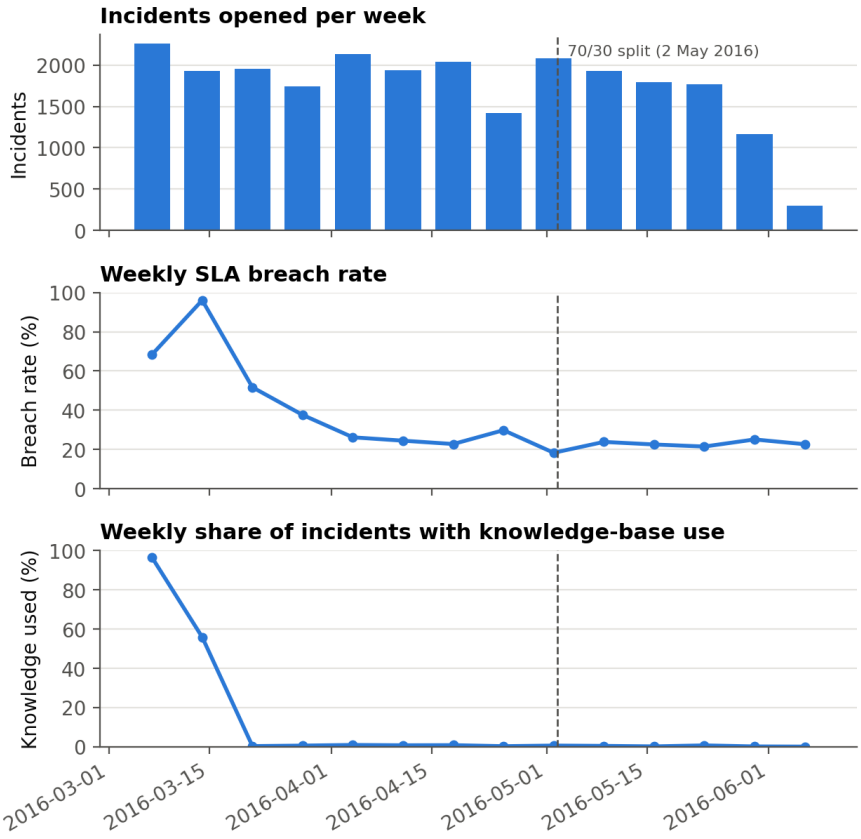


Note. Bars give the percentage of incidents that breached their SLA in each group. Data from Amaral et al. (2018b).

A structural break in March 2016. Weekly volume is steady, but the breach rate exceeds 68% in early March and falls to 18% to 30% from April, while the knowledge flag drops from 97% of incidents to below 1% (Figure 6). Before 21 March, 58.6% of 5,906 incidents carry the flag and 73.1% breached; afterwards, 0.5% carry it and 25.2% breached. Knowledge use was most likely recorded differently from mid-March, during a backlog. The flag therefore marks the period, confounding RQ3, and the RQ4 training and test periods differ in breach rate (42.3% and 23.3%); both issues are handled below.

Figure 6

Incidents per Week, Weekly Breach Rate, and Weekly Share of Knowledge-Base Use



Note. Each week is labelled by its last day; weeks with under 300 incidents are not shown. The dashed vertical line is the 70/30 time split used in RQ4.

Research Hypotheses

Each research question is tested at the 5% significance level ($\alpha = .05$). H_0 denotes the null and H_a the alternative hypothesis; the subscript gives the research question; β_1 is the logistic regression coefficient of the variable of interest, and e^β_1 is its odds ratio (OR) (Table 3).

Table 3

Null and Alternative Hypotheses by Research Question

RQ	Null hypothesis (H_0)	Alternative hypothesis (H_a)	Statistical method
RQ1	H_{01}: SLA breach is independent of priority	H_{a1}: SLA breach is associated with priority	χ^2 test of independence (4 × 2); Holm-adjusted pairwise tests
RQ2	H_{02}: OR for reassignment count = 1 ($\beta_1 = 0$), given priority and category	H_{a2}: OR ≠ 1 ($\beta_1 \neq 0$)	χ^2 test; multivariable logistic regression (Wald z)
RQ3	H_{03}: OR for knowledge use = 1 ($\beta_1 = 0$), given priority and category	H_{a3}: OR ≠ 1 ($\beta_1 \neq 0$)	χ^2 test; logistic regression; Mantel–Haenszel
RQ4	H_{04}: AUC of best model = AUC of priority-only baseline	H_{a4}: AUC of best model > AUC of baseline	DeLong test for correlated AUCs (one-sided)

Minimum Sample Size

Minimum sample sizes were set in advance with $\alpha = .05$, power = 0.80, and Cohen's (2013) small effect $w = 0.10$. For a χ^2 test, $N = \lambda / w^2$, where λ is the noncentrality parameter giving 80% power at the test's degrees of freedom (Faul et al., 2007); for RQ2 and RQ3, N is divided by $(1 - R^2)$, the variance in the variable of interest explained by the controls. For RQ4, the test set must estimate an AUC of $A = 0.75$ within ±0.03, with standard error (SE) from Hanley and McNeil (1982): $SE^2 = [A(1 - A) + (n_1 - 1)(Q_1 - A^2) + (n_0 - 1)(Q_2 - A^2)] / (n_1 n_0)$, $Q_1 = A / (2 - A)$, $Q_2 = 2A^2 / (1 + A)$, increasing n until $1.96 \times SE \leq 0.03$ and dividing by the 30% test share (Table 4; Appendix A).

Table 4

Minimum Sample Size by Research Question

RQ	Method	Parameters	Computation	Minimum N
RQ1	χ^2 power, 4 × 2	w = 0.10; df = 3; λ = 10.903	10.903 / 0.10^2 = 1,090.3	1,091
RQ2	χ^2 power, 2 × 2, adjusted	w = 0.10; df = 1; λ = 7.849; R^2 = 0.047	785 / (1 − 0.047) = 823.7	824
RQ3	χ^2 power, 2 × 2, adjusted	w = 0.10; df = 1; R^2 = 0.154	785 / (1 − 0.154) = 927.9	928
RQ4	AUC precision (Hanley & McNeil)	A = 0.75; ±0.03; prevalence 0.366; 30% test	test *n* = 1,191; 1,191 / 0.30	3,970
All	*N* = max(RQ1–RQ4)	Available: 24,918 incidents	max(1,091; 824; 928; 3,970)	3,970

Note. α = .05 and power = 0.80 throughout. R^2 values were observed in the data. Results are rounded up.

RQ4 is binding, so *N* = 3,970; the 24,918 incidents are 6.3 times this, so all were used. The smallest expected cell in the RQ1 table is 98.8, and although the test breach rate (23.3%) is below the assumed rate, the 7,476 test incidents are 4.4 times the 1,681 then required, giving a 95% confidence half-width of about ±0.012 for the best AUC.

Statistical Methods

RQ1 applies the χ^2 test of independence with Cramér's V, since at *N* = 24,918 trivial differences are significant, and Holm (1979) correction for pairwise tests. Because reassignment and knowledge use relate to priority and category, RQ2 and RQ3 fit logistic regressions, logit $P(Y = 1) = \beta_0 + \beta_1 X + \Sigma\gamma_k$(priority, category), reporting the adjusted odds ratio e^β_1 with a Wald z test (Hosmer et al., 2013); categories under 30 incidents are merged into 31 groups. Robustness checks use Mantel–Haenszel pooling with the Breslow–Day test (Mantel & Haenszel, 1959), the week of opening, banded reassignment with a reassignment × priority interaction (likelihood-ratio test), and separate pre- and post-break estimates for knowledge. RQ4 uses the one-sided DeLong et al. (1988) test.

Database Set-Up and Data Access

The notebooks read the CSV files with pandas. For deployment, src/build_database.py loads them into a SQLite database, data/db/itsm.sqlite, as a stand-in for a ServiceNow reporting replica (Table 5): "?" is stored as NULL, opening time and assignment group are indexed, and a view, v_open_queue, joins each incident to its latest risk score. The model, batch scorer, application programming interface (API), and dashboard read data through structured query language (SQL), for example SELECT * FROM incidents WHERE opened_at >= ? AND opened_at < ?. The 58 MB database rebuilds in about four seconds and is not stored in GitHub.

Table 5

SQLite Database Tables and Views

Object	Rows	Content	Written / read by
event_log (table)	141,712	Raw UCI event log, 36 columns, "?" stored as NULL	build_database.py / audit queries
incidents (table)	24,918	One row per incident: first-record and final-record fields, sla_breach	build_database.py / train_model.py, score_batch.py
risk_scores (table)	7,428	Incident number, time scored, model version, probability, band, action	score_batch.py / dashboard, audit queries
v_open_queue (view)	7,428	Incidents joined to risk scores, ordered by risk	Dashboard (app/dashboard.py)

Note. Row counts for risk_scores and v_open_queue reflect the replay of incidents opened in May 2016 (notebook 06).

Data Splitting

A random split would train on tickets logged after those tested and mix the two regimes, so incidents were sorted by opening time and split 70/30: the earliest 17,442 (before 11:30 on 2 May 2016, breach rate 42.3%) train the models and the latest 7,476 (23.3%, mostly May 2016) test them once. Hyperparameters were tuned by five-fold stratified cross-validation (CV) on the training set, and a five-fold forward-chaining CV, validating each fold on the next 2,907 incidents, preserves time order (Bergmeir & Benítez, 2012). For the deployed model, the first

80% of the training period (13,953 incidents) fit the forest and the latest 20% (3,489) fit the recalibration.

Model Selection and Justification

Model 1, priority-only logistic regression (LR), reproduces the current rule and is the benchmark. **Model 2, regularised LR,** is interpretable through odds ratios, and its L1 or L2 penalty controls overfitting across 255 encoded columns. **Model 3, random forest** (Breiman, 2001), averages many trees and captures interactions without specifying them. **Model 4, XGBoost** (Chen & Guestrin, 2016), builds trees sequentially to correct earlier errors. Tree ensembles were preferred to DL because the at-logging data are one row of mostly categorical fields, where trees match or beat neural networks (Grinsztajn et al., 2022), and sequence models such as LSTMs (Tax et al., 2017) need event histories that do not yet exist. Balanced class weights handle imbalance. The pre-set selection rule was the highest test AUC, with validation-to-test stability as tiebreaker.

Training and Test Accuracy and Comparison With Current Methods

Table 6 compares training and test performance. The current rule reaches a test AUC of 0.552; its 0.781 accuracy is the highest only because it flags almost nothing (7.0% of breaches at a 0.5 threshold). Regularised LR barely changes from training to test (0.783 to 0.779). Both tree ensembles fit the training data far more closely (about 0.92); the random forest generalises best (0.785) and XGBoost worst (0.738), which is why selection rests on the test set.

Table 6

Training Versus Test Performance of the Four Models

Model	Train AUC	Test AUC	Train accuracy	Test accuracy	Train F1	Test F1	Test Brier
Model 1: Priority-only LR (current rule)	0.533	0.552	0.594	0.781	0.085	0.130	0.240
Model 2: Regularised LR	0.783	0.779	0.705	0.722	0.661	0.523	0.185
Model 3: Random forest	0.924	0.785	0.846	0.729	0.817	0.532	0.181
Model 4: XGBoost	0.922	0.738	0.841	0.696	0.814	0.481	0.195

Note. Training set n = 17,442 (breach rate 0.423); test set n = 7,476 (0.233). Accuracy and F1 use a 0.5 threshold. F1 = harmonic mean of precision and recall; Brier = mean squared error of predicted probabilities (lower is better).

Architecture Diagram/Workflow

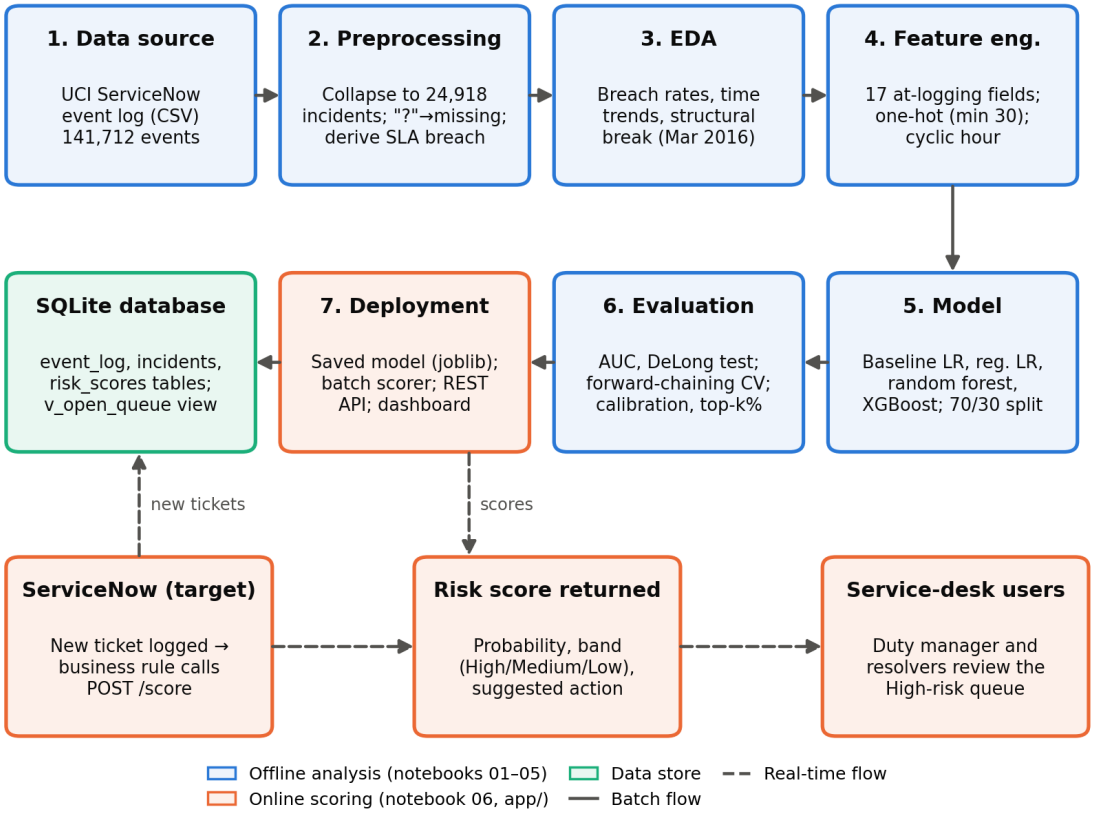
System Overview

Offline, notebooks 01 to 05 prepare and explore the data, engineer features, and train and evaluate the models. Online, notebook 06 and app/ save the chosen model and score tickets in batches or singly through a representational state transfer (REST) API, presenting a ranked queue to users (Figure 7).

Architecture Diagram

Figure 7

End-to-End Workflow of the Proposed System



Note. Blue boxes are offline analysis stages, orange boxes are online scoring components, and the green box is the data store. Solid arrows show batch flow; dashed arrows show the real-time flow when ServiceNow calls the scoring API.

As shown in Figure 7, data moves left to right through preprocessing, EDA, and feature engineering, then right to left through modelling, evaluation, and deployment, which writes scores back to the database. In production, ServiceNow would send each new ticket to the scoring API and display the returned risk to users.

Workflow Components

Data Ingestion

The source is one CSV file of 141,712 event records with 36 columns, mixing identifiers, categorical attributes, Boolean flags, counts, and timestamps. Notebook 01 reads it with pandas, checks the expected shape and the count of 24,918 distinct incidents, and parses timestamps as day-first dates. For deployment, the same file and the processed incident file are loaded into SQLite (Table 5).

Data Preprocessing

Events are ordered by incident, update counter, and update time, so each incident has a unique first and last record. **Missing values.** "?" becomes missing; empty categorical fields at logging, such as symptom (23.0%) and assignment group (3.6%), get their own "Missing" level instead of imputation, because an unrouted ticket is informative; the 1,556 missing resolution times stay missing because resolution time is only descriptive. **Outliers.** The interquartile range (IQR) rule flags 2,848 incidents with more than two reassignments and 812 resolved after 1,000 hours; these real hard cases were kept, with reassignment also banded (0, 1, 2, 3+) and resolution time log-scaled. **Encoding.** One-hot encoding, with levels seen fewer than 30 times in training pooled as "infrequent", gives 255 columns. **Scaling.** Numeric features are standardised for LR; trees use raw values, since scaling does not alter their splits.

Exploratory Data Analysis

Notebook 02 produces the summary statistics and Figures 3 to 6. EDA established the polarity of the outcome, the concentration of breaches in Moderate priority, the dose–response pattern of reassignment, and the March 2016 break, which led directly to the period checks for RQ3 and the time-ordered validation for RQ4.

Feature Engineering

All 17 RQ4 features come from the first event record (Table 7). Six, in four groups, are engineered: the caller's training-period incident count, log-transformed, replacing 5,244 caller codes; hour of logging as sine and cosine, placing 23:00 next to 00:00; day of week with a weekend flag; and a missing-configuration-item flag. Knowledge use is excluded because it marks the period, and is tested in a sensitivity model.

Table 7

Feature Set for the Predictive Model (RQ4)

Feature	Original / engineered	Encoding	Reason for inclusion
Priority	Original	One-hot	Current rule; only feature of Model 1
Impact, urgency	Original	One-hot	Finer detail than priority
Category, subcategory, symptom	Original	One-hot, rare pooled, Missing level	Service area and reported problem
Assignment group (first)	Original	One-hot, rare pooled, Missing level	Routing quality at logging
Location, contact type, notify, priority confirmed	Original	One-hot / binary	Context of the request
Caller frequency	Engineered	log(1 + n), training data only	Frequent callers differ; avoids 5,244 dummies
Hour (sine, cosine), weekend, day of week	Engineered	Numeric / binary / one-hot	Daytime queues breach more
Configuration item missing	Engineered	Binary	Data-quality signal

Note. Excluded to prevent leakage: reassignment, reopen, and update counts; resolution and closure times; closure code; final-record priority.

Model Development

Each model is a scikit-learn pipeline combining encoding and estimation, so identical transformations apply to training, test, and new data. Grid search with five-fold stratified CV on AUC selected C = 1 with an L1 penalty for LR; a minimum leaf of 2 with √p features per split for the random forest (400 trees); and depth 7 with learning rate 0.1 for XGBoost (400 trees, 80% row and column sampling). All runs use seed 42.

Model Evaluation

Test-set AUC, the quantity in H_{04}, is the primary metric, compared by the DeLong test. Precision–recall AUC (PR-AUC), precision, recall, and F1 describe the breach class, and the Brier score measures probability accuracy. Models are also compared at capacity thresholds, flagging the top 10%, 20%, or 30% of tickets by risk, and calibration is checked by decile.

Deployment

Notebook 06 rebuilds the database, trains and saves the deployable model with joblib, writes batch scores to risk_scores, and tests the FastAPI service (POST /score, GET /health); the Streamlit dashboard reads the scored queue (Appendix C).

Tools and Technologies

Table 8 lists the software and computing environment. All analysis ran on a CPU; no GPU was used or needed, because tree ensembles on 24,918 rows train in seconds.

Table 8

Tools, Libraries, and Computing Environment

Layer	Tool (version)	Use
Language	Python 3.11 or later	All analysis and deployment code
Data handling	pandas 3.0.2, NumPy 2.4.4	Loading, collapsing, feature engineering
Statistics	SciPy 1.17.1, statsmodels 0.15.0	χ^2 tests, logistic regression, Mantel–Haenszel
ML	scikit-learn 1.8.0, XGBoost 3.2.0	Pipelines, LR, random forest, CV, metrics; boosting
Visualisation	Matplotlib 3.10.9	All figures
Database	SQLite (Python sqlite3)	event_log, incidents, risk_scores
Deployment	joblib, FastAPI, Uvicorn, Streamlit	Saved model, REST API, dashboard
Compute	Google Colab standard CPU runtime; Windows laptop (CPU only)	Notebook execution; no GPU
Version control	Git and GitHub	Code, data, results, reports

Note. Exact versions are pinned in requirements.txt. Each notebook runs in under ten minutes on a CPU.

Results

All results come from notebooks 03 to 06, and every number can be regenerated from the repository. Intervals are 95% confidence intervals (CI), and p-values below .001 are reported as $p < .001$.

Model Performance

Table 9

Model Performance Comparison (Temporal Test Set, n *= 7,476)*

Model	Features used	Validation method	CV AUC	Test AUC [95% CI]	PR-AUC	Precision / recall at top 20%	Key observation
Model 1: Priority-only LR	Priority only	Temporal 70/30	0.533	0.552 [0.545, 0.558]	0.289	0.302 / 0.259	Current rule; barely above chance
Model 2: Regularised LR	17 at-logging	Temporal 70/30; 5-fold CV	0.768	0.779 [0.768, 0.791]	0.532	0.536 / 0.460	Close second; most stable over time
Model 3: Random forest (best)	17 at-logging	Temporal 70/30; 5-fold CV	0.790	**0.785 [0.773, 0.796]**	0.536	0.542 / 0.466	Highest test AUC and PR-AUC
Model 4: XGBoost	17 at-logging	Temporal 70/30; 5-fold CV	0.787	0.738 [0.725, 0.751]	0.481	0.477 / 0.410	Largest drop from CV to test

Note. CV AUC = mean five-fold stratified cross-validated AUC in the training set. CI = DeLong 95% confidence interval. Top 20% = the 1,495 test incidents with the highest predicted risk; chance precision is 0.233. PR-AUC chance level is 0.233.

The random forest is the best model, with the highest test AUC (0.785) and PR-AUC (0.536) and almost no drop from CV to test (Table 9). It beats the priority rule by 0.233 (DeLong $z = 38.45$, one-sided $p < .001$). Regularised LR is a close, non-significant second (difference 0.005, $z = 1.66$, two-sided $p = .097$); both beat XGBoost significantly (random forest by 0.047, $z = 11.76$, $p < .001$).

Forward-chaining CV confirms the ranking (Table 10). The fold validating on 19 to 30 March, during the break, is weakest for every model, and XGBoost is the weakest ML model in every fold. Over the five folds, regularised LR (0.722) and random forest (0.713) are close and far above the priority rule (0.548); in the latest fold both reach 0.788.

Table 10

Forward-Chaining Cross-Validation AUC Within the Training Period

Fold (validation period, breach rate)	Priority LR	Regularised LR	Random forest	XGBoost
1 (9–19 Mar 2016, 77.9%)	0.561	0.704	0.705	0.660
2 (19–30 Mar 2016, 35.1%)	0.527	0.600	0.556	0.545
3 (30 Mar–8 Apr 2016, 25.5%)	0.531	0.751	0.744	0.682
4 (8–20 Apr 2016, 23.2%)	0.538	0.768	0.772	0.687
5 (20 Apr–2 May 2016, 22.0%)	0.582	0.788	0.788	0.759
Mean (SD)	0.548 (0.023)	0.722 (0.075)	0.713 (0.093)	0.667 (0.077)

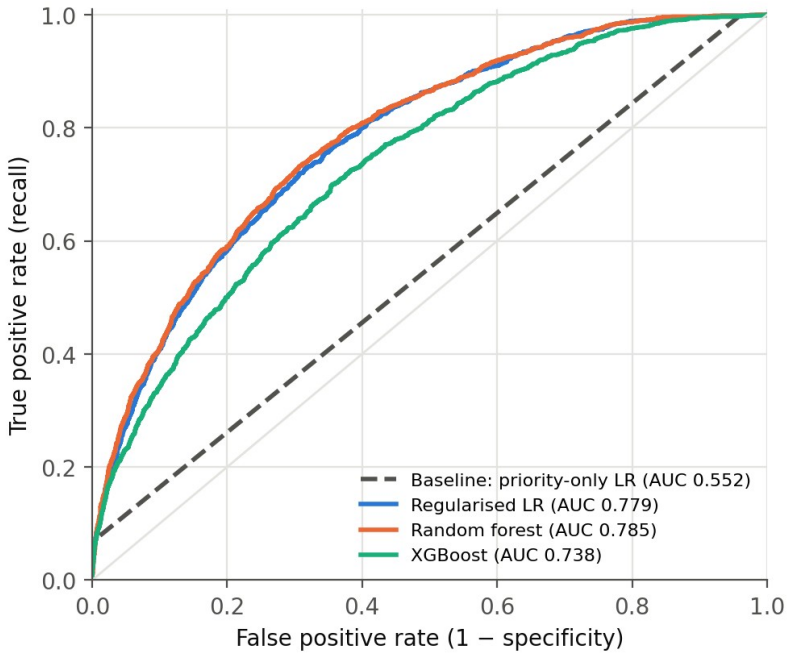
Note. Each fold trains on all earlier training incidents and validates on the next 2,907 incidents; hyperparameters are fixed at the values selected in Table 9. SD = standard deviation.

Visual Evidence

Figure 8 shows the ROC curves. The priority rule hugs the diagonal, because 95% of incidents share the same priority at logging, while the three ML models dominate it at every false-positive rate.

Figure 8

Receiver Operating Characteristic Curves on the Time-Ordered Test Set

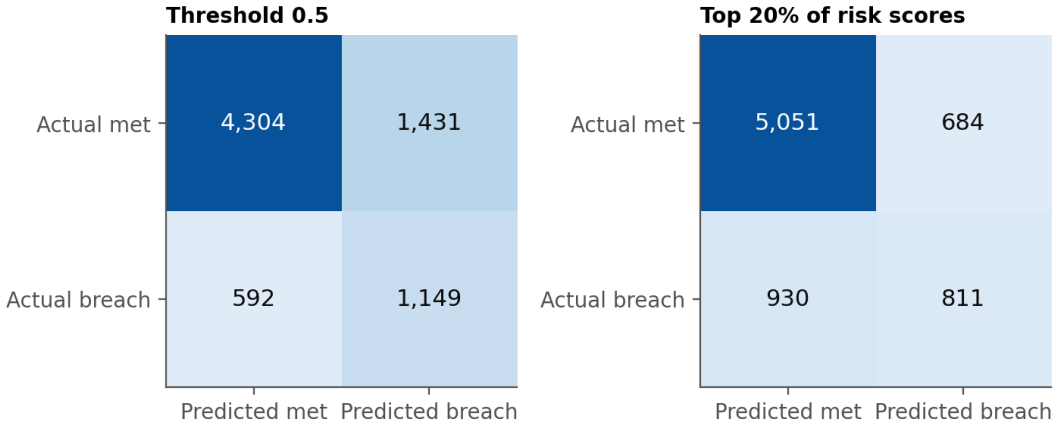


Note. The test set holds the 7,476 incidents opened from 11:30 on 2 May 2016. The baseline (priority only) is dashed, and the pale diagonal marks chance performance (AUC = 0.5).

Figure 9 shows the random forest's confusion matrices. At the default threshold of 0.5 it flags 2,580 tickets and catches 1,149 of 1,741 breaches (recall 0.660, precision 0.445). A service desk is more likely to review a fixed share; flagging the top 20% (1,495 tickets) catches 811 breaches with a precision of 0.542, so more than half the reviewed tickets are real risks, against 23.3% by chance.

Figure 9

Confusion Matrices of the Random Forest at Two Operating Points

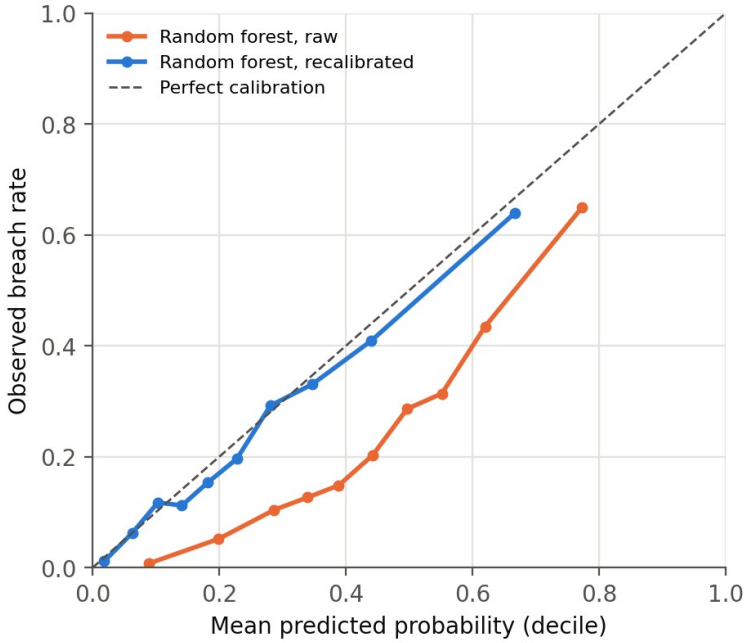


Note. Left: predicted breach when the score is at least 0.5. Right: predicted breach for the 20% highest-risk test incidents. Positive class = breach.

Figure 10 shows calibration. The raw forest predicts a mean breach probability of 0.419, near the training rate, while 23.3% of test incidents breached. After Platt recalibration on the last two training weeks, the curve follows the diagonal, the mean prediction is 0.247, and the Brier score falls from 0.181 to 0.146, below the 0.179 of a constant forecast (Niculescu-Mizil & Caruana, 2005). This matters because the dashboard reports expected breaches.

Figure 10

Calibration of the Random Forest Before and After Recalibration

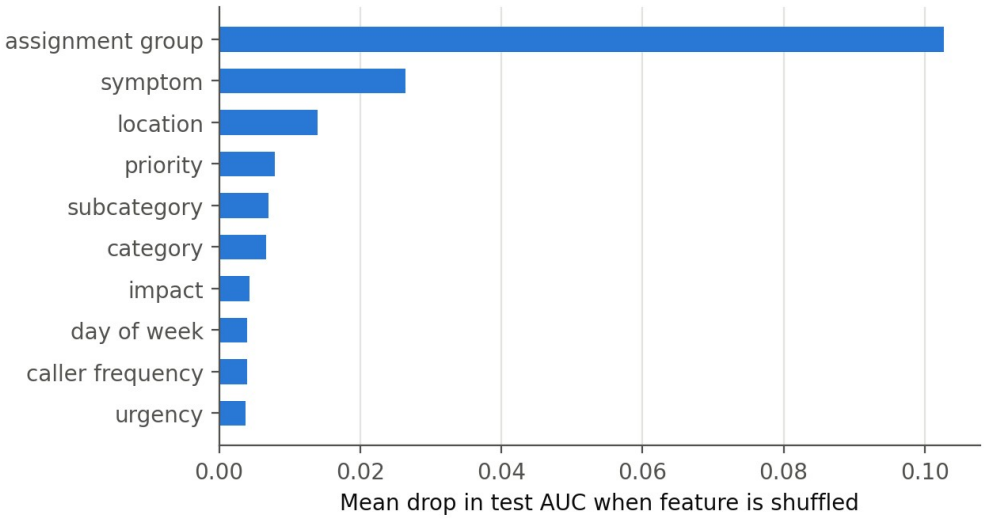


Note. Each point is one decile of predicted probability on the test set. Recalibration uses a logistic (Platt) map fitted on incidents opened 18 April to 2 May 2016.

Figure 11 ranks the features by permutation importance. The first assignment group stands out: shuffling it cuts test AUC by 0.103, compared with 0.026 for symptom, 0.014 for location, and 0.008 for priority. The group a ticket is first sent to matters much more than how urgent it is said to be.

Figure 11

Ten Most Important Random-Forest Features (Permutation Importance)



Note. Each bar is the average fall in test AUC across five random permutations of that feature.

Results by Research Question

Table 11 summarises the hypothesis tests and decisions at α = .05.

Table 11

Hypothesis Test Results and Decisions (α = .05)

RQ	Test	Result	Effect size	Decision
RQ1	χ^2 test of independence, priority × breach	$\chi^2(3, N = 24{,}918) = 1{,}293.1$, $p < .001$	$V = 0.228$	Reject H_{01}
RQ2	χ^2 test, reassigned × breach	$\chi^2(1, N = 24{,}918) = 2{,}868.4$, $p < .001$; OR = 4.33 [4.10, 4.58]	φ = 0.339	
	Logistic regression: reassignment count + priority + category	OR = 1.95 [1.90, 2.00] per reassignment, $z = 48.46$, $p < .001$	McFadden $R^2 = 0.217$	Reject H_{02}
RQ3	χ^2 test, knowledge × breach	$\chi^2(1, N = 24{,}918) = 2{,}399.9$, $p < .001$; OR = 6.24 [5.76, 6.76]	φ = 0.310	
	Logistic regression: knowledge + priority + category	OR = 6.62 [6.05, 7.25], $z = 41.01$, $p < .001$	McFadden $R^2 = 0.171$	Reject H_{03}
	Mantel–Haenszel across the two periods (robustness)	OR = 1.17 [1.05, 1.31], $p = .006$		Effect mostly period
RQ4	DeLong test, random forest vs priority LR (one-sided)	ΔAUC = 0.233, $z = 38.45$, $p < .001$	AUC 0.785 vs 0.552	Reject H_{04}

Note. OR = odds ratio with 95% confidence interval in brackets; φ = phi coefficient; V = Cramér's V. Category controls use 31 groups.

RQ1: priority. Priority and breach are clearly related, $\chi^2(3, N = 24{,}918) = 1{,}293.1$, $p < .001$, $V = 0.228$, which is a small-to-medium effect. After Holm correction, five of the six pairs of priorities still differ; the exception is Critical against High (98.1% and 99.5%, adjusted $p = .086$). H_{01} is rejected. The direction is the reverse of what priority is meant to achieve: the most urgent incidents are the ones that most often miss their targets, which suggests the targets for those levels are tighter than the process can meet.

RQ2: reassignment. With priority and category held constant, each additional reassignment multiplies the odds of breach by 1.95 (95% CI 1.90 to 2.00, $z = 48.46$, $p < .001$), with a variance inflation factor of only 1.05. H_{02} is rejected. The effect is not linear (Table 12): against no reassignment, the odds are 2.88, 5.89, and 12.78 times higher after one, two, and three or more, so the first transfer is the largest step. It also varies by priority (likelihood-ratio $\chi^2(3) = 15.0$, $p = .002$), being largest for Moderate and about half that for Low tickets, while Critical and High tickets breach 98% to 100% of the time at every count. Controlling for week of opening barely changes it (OR 1.93).

RQ3: knowledge-base use. In the pre-specified model, knowledge use carries much higher odds of breach (OR 6.62, $p < .001$), so H_{03} is formally rejected, but the robustness checks show this is mostly a period effect (Table 12). Estimated separately, the adjusted odds ratio is 1.15 (95% CI 1.01 to 1.31, $p = .035$) before 21 March 2016 and 1.17 (0.74 to 1.86, $p = .51$) afterwards, with a Mantel–Haenszel estimate of 1.17 across periods, while controlling for week reverses the sign (OR 0.61). The defensible conclusion is that this log does not support a material effect of knowledge-base use on SLA breach in either direction.

Table 12

Robustness Checks for RQ2 and RQ3

Check	Estimate [95% CI]	Interpretation
RQ2: banded reassignment (vs 0): 1 / 2 / 3+	OR 2.88 / 5.89 / 12.78	First hand-off is the largest step
RQ2: bands within Moderate priority	OR 2.92 / 5.95 / 13.17	Strongest where volume is
RQ2: bands within Low priority	OR 1.51 / 3.26 / 4.26	Weaker; interaction $p = .002$
RQ2: adjusted for opening week	OR 1.93 [1.88, 1.99] per reassignment	Robust to the March break
RQ3: before 21 March 2016 (n = 5,906)	OR 1.15 [1.01, 1.31], $p = .035$	Small positive
RQ3: from 21 March 2016 (n = 19,012)	OR 1.17 [0.74, 1.86], $p = .51$	Not significant; 0.5% use
RQ3: adjusted for opening week	OR 0.61 [0.44, 0.86], $p = .004$	Sign depends on time control

Note. All models adjust for priority and category. The interaction test is restricted to Moderate and Low incidents (n = 24,240) because Critical and High incidents breach almost without exception, which makes interaction terms inestimable.

RQ4: prediction at logging. Using only the 17 attributes known at logging, the random forest reaches a test AUC of 0.785 against 0.552 for the priority-only baseline ($z = 38.45$, one-sided $p < .001$), so H_{04} is rejected; regularised LR and XGBoost also beat the baseline. Adding the knowledge flag lifts the AUC only to 0.788, and using the period after 21 March 2016 alone gives 0.803 against 0.551, so the gain does not rest on the break. The first assignment group carries most of the signal (Figure 11).

Overall Interpretation

The answers fit together. Priority predicts breach only at the extremes and carries no information within Moderate priority, which holds 91% of breaches. Reassignment is the strongest operational driver, and the model points to its root, the first group a ticket is sent to. The knowledge flag reflected a change in recording. Performance is limited mainly by the shift from 42% breaches in training to 23% in testing, which hurt XGBoost and inflated raw probabilities and was handled by time-ordered validation and recalibration.

Practical Significance

At this sample size significance is assured, so the business test is what the model changes. Reviewing the 20% highest-risk tickets, the random forest finds 46.6% of breaches with 54.2% of reviewed tickets being real risks; the priority rule at the same workload finds 25.9% at 30.2%. At about 240 incidents a day, that is a list of about 48 tickets, of which about 26 would otherwise breach. Avoiding the first reassignment cuts a Moderate ticket's odds of breach by about two-thirds (OR 2.92), and RQ1 supports renegotiating Critical and High targets that are missed almost every time.

Implementation and User Benefit

The best model was packaged as a working prototype (notebook 06, src/, and app/) that runs on a laptop and moves to a cloud service without code changes; here it replays May 2016 as if the tickets were new.

Deployment Approach

The deployable model, rf-v1.0, is the Table 9 random forest fitted on the first 80% of the training period with Platt recalibration on the last 20%, saved with joblib as one 25 MB file that includes the caller-frequency table and band thresholds. A ticket is High risk if its score is in the top 20% of the recalibration window, Medium in the next 20%, and Low otherwise. On the 7,476 test incidents it scores an AUC of 0.776, its mean probability (0.247) matches the observed rate (0.233), and the High band holds 22.2% of tickets with a 50.4% breach rate and 48.1% of all breaches. A batch scorer writes scores to risk_scores, and a REST API (FastAPI) scores tickets posted as JavaScript Object Notation (JSON); in production it would run as a container on a managed service such as Azure App Service.

System Integration

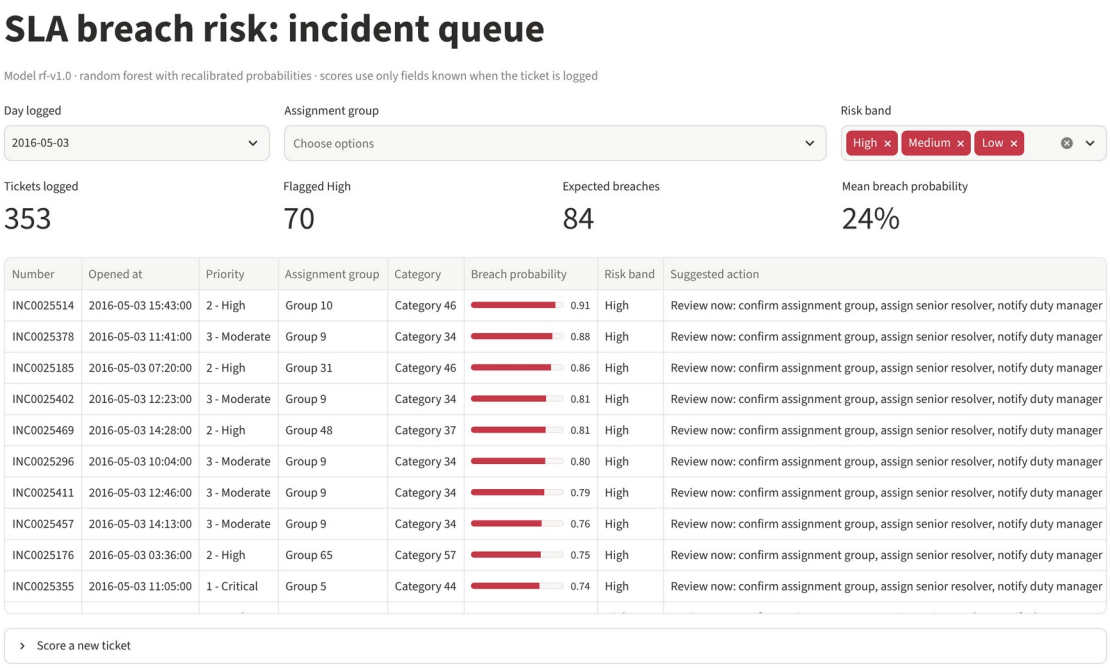
No change is needed to how tickets are logged. An asynchronous ServiceNow business rule on incident insert would send the at-logging fields to POST /score through an outbound REST message and write the returned probability and band to two custom fields, which can drive assignment rules, dashboard filters, or a duty-manager notification. Each score carries the model version and is stored in risk_scores for audit. Where real-time calls are not possible, the batch scorer can run every few minutes against a reporting replica. Retraining reuses src/train_model.py, with a monthly calibration check because process changes can move the base rate quickly.

User Interaction

Users work in the Streamlit dashboard (Figure 12). A duty manager picks a day, filters by assignment group or risk band, and sees four indicators (tickets logged, flagged High, expected breaches, and mean probability) above a risk-ordered queue showing each ticket’s probability, band, and suggested action; a form scores a new ticket from its priority, impact, urgency, group, category, and channel. Other systems call the API with the same fields and receive the probability, band, and action.

Figure 12

Service-Desk Risk Dashboard Showing the Queue for 3 May 2016



Note. Screenshot of app/dashboard.py running locally against the SQLite database after the May 2016 replay. Ticket numbers, groups, and categories are the anonymised codes in the source data.

Benefits to Users

Operational efficiency. Reviewing one ticket in five finds almost half of the breaches, about 1.8 times the yield of the same review by priority. **Financial impact.** Each avoided breach reduces exposure to service credits and review effort, which is material for a desk that breaches about a quarter of its tickets. **Strategic value.** The scores and the RQ1 and RQ2 evidence support renegotiating unrealistic targets, fixing the routing of high-risk groups, and showing clients proactive management.

Example Use Case

On 3 May 2016 the desk logged 353 incidents. At 15:43 a High-priority ticket in Category 46 went to Group 10, and the API returned a breach probability of 0.91, a High band, and the action “confirm assignment group, assign senior resolver, notify duty manager”. By

day's end the dashboard showed 70 High-risk tickets and predicted 84 breaches; 76 occurred, 41 of them among the flagged tickets (a 58.6% hit rate against 21.5% overall). Reviewing those 70 tickets would have reached more than half of the day's breaches while they were still preventable.

Limitations and Further Improvements

The study is designed to be transparent about what the data can and cannot show.

Limitations

Data. The incidents come from one anonymised company, mostly from three months in 2016; names are coded, SLA targets are absent, and there is no free text, ruling out text and LLM methods. The March 2016 recording change altered the breach rate and the meaning of the knowledge flag. **Method.** RQ1 to RQ3 are observational, so odds ratios are associations, not causal effects, and unrecorded factors such as analyst experience or workload may confound them. **Model.** The tree ensembles fit the training period far better than the test period. Scores shift slightly with library versions (in Google Colab, XGBoost scored 0.744 to 0.751 instead of 0.738, the random forest 0.784 instead of 0.785, and the deployed model 0.777 instead of 0.776, with no change to any conclusion); permutation importance can understate correlated features; and target encoding (Micci-Barreca, 2001) remains untested.

Impact of Limitations

Single-organisation data limit generalisability: the coefficients describe this desk, and only the method transfers. The period shift lowered measured accuracy and would recur after any process change, hence monthly recalibration. RQ3 is most affected, because period and knowledge flag cannot be fully separated. Reliability is high: the scripted, seeded, version-pinned pipeline regenerates every number in this report.

Future Improvements

- Collect data from more organisations and recent years, with SLA targets and hand-off timestamps, to test generalisation and model time-to-breach.
- Add target encoding and group × category features, and explain individual scores with SHapley Additive exPlanations (SHAP).
- Update the risk score during a ticket's life with an LSTM or transformer once reassignments occur (Bukhsh et al., 2021; Tax et al., 2017).
- Run a pilot that reviews only half of the High-risk tickets to measure the causal effect on breaches.

Future Scope

The approach extends to other outcomes, such as reopening or escalation, and to service requests and changes. Where tickets contain text, an LLM agent following the ReAct pattern (Yao et al., 2023) could work the High-risk queue by calling the risk API, retrieving similar past incidents and articles, and drafting a routing recommendation, as RCACopilot does for root-cause analysis (Chen et al., 2024), with the calibrated score directing its effort.

References

Ahmed, T., Ghosh, S., Bansal, C., Zimmermann, T., Zhang, X., & Rajmohan, S. (2023). Recommending root-cause and mitigation steps for cloud incidents using large language models. In *Proceedings of the 45th International Conference on Software Engineering* (pp. 1737–1749). IEEE. https://doi.org/10.1109/ICSE48619.2023.00149

Amaral, C. A. L., Fantinato, M., & Peres, S. M. (2018a). Attribute selection with filter and wrapper: An application on incident management process. In M. Ganzha, L. Maciaszek, & M. Paprzycki (Eds.), *Proceedings of the 2018 Federated Conference on Computer Science and Information Systems* (Annals of Computer Science and Information Systems, Vol. 15, pp. 679–682). IEEE. https://doi.org/10.15439/2018F126

Amaral, C., Fantinato, M., & Peres, S. (2018b). *Incident management process enriched event log* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C57S4H

Amaral, C. A. L., Fantinato, M., Reijers, H. A., & Peres, S. M. (2019). Enhancing completion time prediction through attribute selection. In E. Ziemba (Ed.), *Information technology for management: Emerging research and applications* (Lecture Notes in Business Information Processing, Vol. 346, pp. 3–23). Springer. https://doi.org/10.1007/978-3-030-15154-6_1

Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences, 191,* 192–213. https://doi.org/10.1016/j.ins.2011.12.028

Botezatu, M. M., Bogojeska, J., Giurgiu, I., Voelzer, H., & Wiesmann, D. (2015). Multi-view incident ticket clustering for optimal ticket dispatching. In *Proceedings of the 21st ACM*

SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 1711–1720). ACM. https://doi.org/10.1145/2783258.2788607

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review, 78*(1), 1–3. https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

Bukhsh, Z. A., Saeed, A., & Dijkman, R. M. (2021). *ProcessTransformer: Predictive business process monitoring with transformer network* (arXiv:2104.00721). arXiv. https://doi.org/10.48550/arXiv.2104.00721

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., Zeng, J., Ghosh, S., Zhang, X., Zhang, C., Lin, Q., Rajmohan, S., Zhang, D., & Xu, T. (2024). Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems* (pp. 674–688). ACM. https://doi.org/10.1145/3627703.3629553

Cohen, J. (2013). *Statistical power analysis for the behavioral sciences* (2nd ed.). Routledge. https://doi.org/10.4324/9780203771587 (Original work published 1988)

Dang, Y., Lin, Q., & Huang, P. (2019). AIOps: Real-world challenges and research innovations. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion*

Proceedings (ICSE-Companion) (pp. 4–5). IEEE. https://doi.org/10.1109/ICSE-Companion.2019.00023

DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics, 44*(3), 837–845. https://doi.org/10.2307/2531595

Evermann, J., Rehse, J.-R., & Fettke, P. (2017). Predicting process behaviour using deep learning. *Decision Support Systems, 100*, 129–140. https://doi.org/10.1016/j.dss.2017.04.003

Faul, F., Erdfelder, E., Lang, A.-G., & Buchner, A. (2007). G*Power 3: A flexible statistical power analysis program for the social, behavioral, and biomedical sciences. *Behavior Research Methods, 39*(2), 175–191. https://doi.org/10.3758/BF03193146

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in Neural Information Processing Systems 35* (pp. 507–520). Curran Associates. https://doi.org/10.52202/068431-0037

Hanley, J. A., & McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology, 143*(1), 29–36. https://doi.org/10.1148/radiology.143.1.7063747

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics, 6*(2), 65–70. https://www.jstor.org/stable/4615733

Hosmer, D. W., Jr., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3rd ed.). Wiley. https://doi.org/10.1002/9781118548387

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data, 6*(4), 1–21. https://doi.org/10.1145/2382577.2382579

Leitner, P., Michlmayr, A., Rosenberg, F., & Dustdar, S. (2010). Monitoring, prediction and prevention of SLA violations in composite services. In *2010 IEEE International Conference on Web Services* (pp. 369–376). IEEE. https://doi.org/10.1109/ICWS.2010.21

Mantel, N., & Haenszel, W. (1959). Statistical aspects of the analysis of data from retrospective studies of disease. *Journal of the National Cancer Institute, 22*(4), 719–748. https://doi.org/10.1093/jnci/22.4.719

Marrone, M., & Kolbe, L. M. (2011). Impact of IT service management frameworks on the IT organization: An empirical study on benefits, challenges, and processes. *Business & Information Systems Engineering, 3*(1), 5–18. https://doi.org/10.1007/s12599-010-0141-5

Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter, 3*(1), 27–32. https://doi.org/10.1145/507533.507538

Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning* (pp. 625–632). ACM. https://doi.org/10.1145/1102351.1102430

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), Article e0118432. https://doi.org/10.1371/journal.pone.0118432

Shao, Q., Chen, Y., Tao, S., Yan, X., & Anerousis, N. (2008). Efficient ticket routing by resolution sequence mining. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 605–613). ACM. https://doi.org/10.1145/1401890.1401964

Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion, 81*, 84–90. https://doi.org/10.1016/j.inffus.2021.11.011

Tax, N., Verenich, I., La Rosa, M., & Dumas, M. (2017). Predictive business process monitoring with LSTM neural networks. In E. Dubois & K. Pohl (Eds.), *Advanced information systems engineering* (Lecture Notes in Computer Science, Vol. 10253, pp. 477–492). Springer. https://doi.org/10.1007/978-3-319-59536-8_30

Teinemaa, I., Dumas, M., La Rosa, M., & Maggi, F. M. (2019). Outcome-oriented predictive process monitoring: Review and benchmark. *ACM Transactions on Knowledge Discovery from Data*, *13*(2), 1–57. https://doi.org/10.1145/3301300

van der Aalst, W. (2016). *Process mining: Data science in action* (2nd ed.). Springer. https://doi.org/10.1007/978-3-662-49851-4

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629

Appendix A: Sample Size Code

Running this script (src/sample_size.py) regenerates every value in Table 4.

```
# QM640 capstone - minimum sample sizes (alpha = 0.05, power = 0.80)
# Dependent variable for all RQs: SLA breach (binary)
import numpy as np
from scipy.stats import chi2, ncx2

def chi2_n(w, df, a=0.05, p=0.80):            # chi-square test of independence
    n, c = 10, chi2.ppf(1 - a, df)
    while ncx2.sf(c, df, n * w**2) < p:
        n += 1
    return n

def auc_test_n(auc, e, prev):                 # RQ4 (Hanley & McNeil, 1982)
    q1, q2 = auc / (2 - auc), 2 * auc**2 / (1 + auc)
    n = 50
    while True:
        pos = round(n * prev); neg = n - pos
        var = (auc*(1-auc) + (pos-1)*(q1-auc**2) + (neg-1)*(q2-auc**2)) / (pos*neg)
        if 1.96 * np.sqrt(var) <= e:
            return n
        n += 1

rq1 = chi2_n(0.10, 3)                         # priority (4) x breach (2): 1091
rq2 = int(np.ceil(chi2_n(0.10, 1) / (1 - 0.047)))  # 785 / 0.953 -> 824
rq3 = int(np.ceil(chi2_n(0.10, 1) / (1 - 0.154)))  # 785 / 0.846 -> 928
test = auc_test_n(0.75, 0.03, 9115 / 24918)   # observed prevalence 0.366: 1191
rq4 = int(np.ceil(test / 0.30))               # 3970
print(rq1, rq2, rq3, test, rq4, "final N =", max(rq1, rq2, rq3, rq4))
```

Appendix B: Notebook Map and Run Order

Run the notebooks in order after installing requirements.txt; each writes its results to results/ and its figures to figures/.

Table B1

Notebooks, Purpose, and Outputs

Notebook	Purpose	Produces
01_data_preparation.ipynb	Load, validate, collapse to incidents, derive variables	data/processed/incidents.csv
02_cleaning_and_eda.ipynb	Cleaning checks, summary statistics, EDA figures	Figures 3–6; results/eda.json
03_rq1_rq3_hypothesis_tests.ipynb	χ^2 tests, logistic regression, Mantel–Haenszel, week checks	Table 11 (RQ1–RQ3); results/rq1_rq3.json
04_rq4_prediction_models.ipynb	Tuning, test evaluation, DeLong test, importance	Table 9; Figures 8, 11; results/rq4.json
05_extended_evaluation.ipynb	Train vs test, forward-chaining CV, calibration, top-k%, RQ2/RQ3 robustness	Tables 6, 10, 12; Figures 9, 10; results/final_eval.json
06_deployment_prototype.ipynb	Database, saved model, batch scoring, API test	Table 5; results/deployment.json
src/sample_size.py	Minimum sample size	Table 4

Appendix C: Running the Deployment Prototype

From the repository root, after installing requirements.txt:

```
python src/build_database.py          # builds data/db/itsm.sqlite
python src/train_model.py              # saves models/sla_breach_rf.joblib
python src/score_batch.py --start 2016-05-02 --end 2016-06-01
uvicorn app.api:app --port 8000        # REST API; docs at http://localhost:8000/docs
streamlit run app/dashboard.py         # dashboard at http://localhost:8501

Example request body for POST /score:
[{"number": "INC_DEMO", "opened_at": "2016-05-03 09:15:00",
  "log_priority": "3 - Moderate", "log_impact": "2 - Medium", "log_urgency": "2 -
Medium",
  "log_assignment_group": "Group 70", "log_category": "Category 26",
"log_contact_type": "Phone"}]

Response:
[{"number": "INC_DEMO", "breach_probability": 0.303, "risk_score_raw": 0.528,
  "risk_band": "Medium", "suggested_action": "Watch: check at next queue review"}]
```